

MedAssist AI

Frontend & Data Visualization Milestone 2 Report

Prepared By: AKANKSHA MUMMANA

B.Tech – Computer Science & Engineering

Role: Frontend Developer & Data Visualizer

Internship: Infosys Springboard Internship

Project: MedAssist AI – AI Powered Healthcare Prediction System

Table of Contents

Section	Page
1. Project Description	1
2. Technologies Used	2
3. Environment Setup	3
4. Frontend Architecture	4
5. Landing Page Development	5
6. Authentication Module	6
7. Dashboard Module	7
8. Symptom Checker Module	8
9. AI Prediction Module	9
10. Reports Module	10
11. Profile Management	11
12. Challenges & Solutions	12
13. Conclusion	13

1. Project Description

MedAssist AI is an AI-powered healthcare web application designed to assist users in identifying possible diseases based on the symptoms they provide. The application offers a simple, responsive, and user-friendly interface that enables users to enter their symptoms, view AI-generated disease predictions, access previous reports, and manage their personal health profile.

For this milestone, my primary responsibility as a Frontend Developer & Data Visualizer was connecting our React components to the live backend API services and refactoring our core symptom selection forms. Based on user-experience reviews, I overhauled the Symptom Checker interface to group symptoms into structured medical specialty categories, replacing the long flat checkbox grid.

This milestone successfully achieved live API integrations (connecting user login, prediction history, and analytics endpoints) and implemented key UI layout optimizations. The system now supports categorized specialty tab filtering, tab badge selection counts, and print-friendly layouts for clinical report downloads.

Objectives of this Milestone

- Connect React frontend pages to live FastAPI backend endpoints.
- Overhaul the Symptom Checker module to group symptoms into specialty categories.
- Implement horizontal tab navigation with active select indicators.
- Create dynamic badge counts showing selected symptoms per category.
- Integrate live AI prediction API endpoints and historical analytics.
- Configure custom print CSS layouts for saving reports as clean PDFs.

2. Technologies Used

The MedAssist AI frontend was developed using modern web technologies to ensure scalability, responsiveness, and maintainability.

Technology	Purpose
React.js Hooks	Managing category selection tabs and checked badge updates
Axios / Fetch API	Connecting authentication, prediction, and history API endpoints
Chart.js	Updating dashboard statistics dynamically from live API analytics
Tailwind CSS	Designing specialty tabs navigation and responsive grids
CSS Print Stylesheets	Configuring page break layouts for print-to-PDF formatting
Visual Studio Code	Source code editor
Git & GitHub	Version control and code releases

3. Environment Setup

Development Environment: Visual Studio Code, Node.js & npm, Axios, Git & GitHub

Commands used:

```
npm install  
npm run dev
```

4. Frontend Architecture

Implemented modular folder structure: components/ pages/ services/ styles/ layout/

A reusable Layout component is implemented with Sidebar Navigation and a shared UI frame to ensure consistency across the application.

5. Landing Page Development

Implemented a fully responsive landing page including Hero, About, Features, How It Works, Testimonials, and Footer sections. The design is styled with Tailwind CSS utility classes and includes a modern healthcare illustration integrated into the layout.

6. Authentication Module

Designed secure, responsive Login and Registration pages. The structure is configured to connect with JWT-based backend endpoints, storing user session keys upon authentication.

7. Dashboard Module

Updated the Dashboard module to retrieve statistics dynamically from the `/api/analytics` endpoint. The Chart.js bar and line graphs now pull real-time clinical assessment counts and top symptom frequencies from the backend database.

8. Symptom Checker Module

Overhauled the Symptom Checker interface with **4 Major Contributions** based on usability reviews:

- **Medical Specialty Categorization**: Grouped the 117 symptoms dynamically into distinct categories (such as Cardiology, ENT, Pulmonology, Gastroenterology, Mental Health, General) instead of showing them all flatly.
- **Active State Selection Tab Bar**: Added an interactive horizontal navigation bar at the top of the grid. Clicking a category filter pill instantly swaps the active checkbox grid below.
- **Category Checked Badge Counts**: Added real-time numeric badges to tab headers (e.g. 'ENT (2)'). If a user checks a symptom and jumps to another category tab, they still see how many symptoms are active in other sections.
- **Smart Search Override & User-Friendly Labels**: Global text search now automatically hides tabs and displays search results flatly. Polished the checked chips at the top to display human-readable names (e.g., 'Shortness of Breath') instead of database keys.

9. AI Prediction Module

Connected the prediction result UI to the live `/api/history/check` endpoint. It displays the AI-predicted primary disease, prediction confidence percentages, differential diagnosis weights, severity analysis, and precautions.

10. Reports Module

Connected the patient reports table to the live `/api/history` endpoint to retrieve previous assessment history. I implemented custom print CSS stylesheets (`@media print`) that hide navigation sidebars and headers, allowing patients to save and print report cards as formatted, clean medical documents.

11. Profile Management

Connected profile updates to live backend APIs to persist user settings in the user profile database, supporting real-time profile picture uploads.

12. Challenges & Solutions

Challenges Faced

- Grouping the 117 symptoms dynamically in the frontend without slowing down page load times or blocking the UI thread.
- Generating clean print layouts that look like formal medical documents, as default print triggers included navigation sidebars and buttons.

Solutions Implemented

- Implemented javascript `.reduce()` grouping and cached the categorized list, running grouping operations only when the symptoms are first loaded.
- Configured custom ``@media print`` rules to hide UI chrome (sidebar, navbar, buttons) and style standard document containers with gray borders and text grids.

13. Conclusion

The Frontend Development and Data Visualization Milestone 2 objectives have been completed successfully. All frontend modules are connected to live backend API endpoints, displaying real-time predictions and statistics. The Symptom Checker was overhauled to support categorized tab navigation with active select counts, significantly improving user accessibility. Print-to-PDF styles were integrated to allow clean report exporting. The application is now fully integrated, responsive, and ready for deployment.

Outcome

- Connected all React pages to live backend authentication and prediction APIs.

- Overhauled Symptom Checker with dynamically categorized specialty tabs.
- Implemented tab selection active highlights and checked badge counts.
- Added text search override and formatted display chips for symptoms.
- Configured `@media print` CSS layouts for clean medical report printing.